500 HTTP Status Code in ASP.NET Core Web API

Back to ASP.NET Core Web API Tutorial

500 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **How to Return the Well Internal Server Error Status Code in ASP.NET Core Web API** Applications and Tutorials. Please read our previous article on **Configuring the Browser HTTP Referrer Header in ASP.NET Core Web API** with Examples.

What is a HTTP Status Code?

HTTP status codes are sent from the Web server to Internal Server Error, a generic error message indicating that the server encountered an unexpected condition that prevented it from fulfilling the request. The error is a catch-all. Microsoft IIS uses numerous extensions to the standard set of HTTP status codes to indicate ...

What are the Common Causes of a 500 HTTP Status Code in ASP.NET Core Web API?

A 500 Internal Server Error means that an unexpected server-side error occurred, preventing the API from processing the request. The following are the most common causes of a 500 Internal Server Error in ASP.NET Core Web API:

- **Unhandled Exceptions:** This misstatement if your application that is not caught and handled can result in a 500 error. The server-side code encounters an error, and the framework returns a generic 500 response to the client to avoid exposing sensitive ...
- **Configuration Issues:** Problems with the application's configuration files or environment settings such as incorrect connection strings, missing dependencies, or improper settings can cause the application to fail when it attempts to load or use these configurations. The server may return a 500 error if it encounters ...
- **Resource Exhaustion:** Problems connecting to the database or issues with executing queries can lead to a 500 error. The server may return a generic error message, the database error might be logged on the server, but the client ...
- **Middleware Issues:** If there is a problem in the middleware pipeline that is not properly handled, it can lead to a 500 error. Custom middleware or third-party middleware can introduce errors that affect the ...
- **Dependency Failures:** If external APIs or services that the application depends on are down or return unexpected Server Errors ...
- **Resource Exhaustion:** The server running out of resources such as memory or disk space can also lead to 500 errors ...

How to Return a 500 HTTP Status Code in ASP.NET Core Web API?

In ASP.NET Core Web API, we can return the Well Internal Server Status Code from our controller actions to indicate that something went wrong on the server side. This is useful when we need to handle exceptions or situations where the server encounters an unexpected condition that prevents it from fulfilling the request. Here, we can return a 500 status code in various ways, such as using the StatusCode method or the convenience shortcuts like the HTTP Status Code ...

Throwing an Exception

ASP.NET Core will automatically return a 500 status code when an unhandled exception occurs. For a better understanding, consider the following Controller Action. Note that this method will always throw an exception ...

```
namespace ProductCatalogueWebAnalytics.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ExamplesController : ControllerBase
    {
        [HttpGet("throw-exception")]
        public IActionResult ThrowException()
        {
            // This will always throw an exception
            int[] numbers = new int[3];
            int value = numbers[5];
            return Ok("This will never reach this line");
        }
    }
}
```

Now run the application and navigate the browser to the URL http://localhost:5000/api/Examples/throw-exception and hit enter. You will see the following response.

```
Status: 500 Internal Server Error
Content-Type: text/plain; charset=utf-8
Date: Wed, 12 Feb 2025 05:42:15 GMT
Server: Kestrel
Transfer-Encoding: chunked

System.IndexOutOfRangeException: Index was outside the bounds of the array.
   at ProductCatalogueWebAnalytics.Controllers.ExamplesController.ThrowException() in C:\Projects\ProductCatalogueWebAnalytics\Controllers\ExamplesController.cs:line 15
   at Microsoft.AspNetCore.Mvc.Infrastructure.ActionMethodExecutor.SyncObjectResultExecutor.Execute(IActionResult, IfaceMapper, Object, Object[])
   at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionFunction`1.Execute(ActionContext, IfaceMapper, Int32, Object[])
   at Microsoft.AspNetCore.Mvc.Infrastructure.ActionMethodExecutor.Execute(ActionContext, IfaceMapper, Int32, Object[])
   at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeAction(ActionContext, IfaceMapper, Int32, Object[])
   at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeAsync()
   at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.InvokeAsync()
   at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.Invoke
```